Lista de Exercícios: Critérios de Teste Caixa-Branca
GFC, Line Coverage e Branch Coverage

Prof. André Takeshi Endo

- Para todos os exercícios a seguir, elabore casos de teste em JUnit e Mockito de forma a alcançar 100% dos critérios de teste caixa-branca (estrutural) apresentados.
- Desenhe o grafo de fluxo de controle (GFC) para os métodos.
- Comente os casos de teste com o caminho que foi gerado de acordo com o GFC.
- Use a funcionalidade de cobertura de código de sua IDE para confirmar que os critérios de fluxo de controle foram adequadamente cobertos.

(Exercício 1) Execute as atividades com as classes e métodos implementados até o momento durante a disciplina.

(Exercício 2) Considere os métodos a seguir.

```
public int test01(int a)
{
    return (a > 0) ? 1 : -1;
}

public int test03(int a)
{
    if(a > 10 || a < -10)
        return 0;
    else
        return 1;
}

public int test02(int a)
{
    switch(a)
    {
    case 1: return 1;
    case 2:
        return 2;
    default:
        return 0;
    }
}
```

(Exercício 3) Considere os métodos a seguir.

```java
// return C = A * B
public static double[][] multiply(double[][] A, double[][] B) {
    int mA = A.length;
    int nA = A[0].length;
    int mB = B.length;
    int nB = A[0].length;
    if (nA != mB) throw new RuntimeException("Illegal matrix dimensions.");
    double[][] C = new double[mA][nB];
    for (int i = 0; i < mA; i++)
        for (int j = 0; j < nB; j++)
            for (int k = 0; k < nA; k++)
                C[i][j] += (A[i][k] * B[k][j]);
    return C;
}
```

(Exercício 4) Realize os exercícios de listas anteriores considerando agora teste caixa-branca.

(Exercício 5) Utilize os projetos fornecidos anexos a esta lista.

(Exercício 6) Considere a classe abaixo:

```java
public class Calculadora {
   /**
    * @param vetor
    * @param inicioInterv deve ser >= 0 e menor que fimInterv
    * @param fimInterv    deve ser >= 0 e maior que inicioInterv
    * @return (i) media dos inteiros do vetor que estão no intervalo [inicioInterv, fimInterv]
    *             (ii) -1 se os parametros forem invalidos
    */
   public float calcularMedia(int vetor[], int inicioInterv, int fimInterv) {
      if (inicioInterv < 0 || fimInterv < 0)
         return -1;

      if (inicioInterv >= fimInterv)
         return -1;

      float soma = 0, n = 0;
      for (int i = 0; i < vetor.length; i++) {
         if(vetor[i] >= inicioInterv && vetor[i] <= fimInterv) {
            soma = soma + vetor[i];
            n++;
         }
      }

      return soma/n;
   }
}
```

Elabore casos de teste em JUnit de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (branch coverage). Desenhe o grafo de fluxo de controle (GFC) para o método e comente cada caso de teste com o caminho que foi executado de

acordo com o GFC. O desenho do GFC pode ser feito no espaço em branco no final da avaliação ou encaminhada como uma imagem junto com o projeto; enumere os nós para identificar corretamente os caminhos.

(Exercício 7) Considere as duas classes e as três interfaces a seguir:

Usuario.java

```
public class Usuario {
   private String nome, email;

   public String getNome() {
      return nome;
   }
   public void setNome(String nome) {
      this.nome = nome;
   }
   public String getEmail() {
      return email;
   }
   public void setEmail(String email) {
      this.email = email;
   }
}
```

UsuarioDAO.java

```
public interface UsuarioDAO {
   public ArrayList<Usuario> getAllUsuarios();
}
```

ServidorDeEmail.java

```
public interface ServidorDeEmail {
   public boolean enviar(String email);
}
```

Criptografia.java

```
public interface Criptografia {
   public String criptografar(String mensagem);
}
```

Emissario.java

```
public class Emissario {
   private UsuarioDAO usuarioDAO;
   private ServidorDeEmail servidorEmail;
   private Criptografia criptografia;

   public Emissario(UsuarioDAO usuarioDAO, ServidorDeEmail servidorEmail) {
      this.usuarioDAO = usuarioDAO;
      this.servidorEmail = servidorEmail;
   }

   public void setCriptografia(Criptografia criptografia) {
      this.criptografia = criptografia;
   }

   public String enviarPara(ArrayList<String> nomes) {
      if(nomes == null)
         return "nomes nao informados";

      ArrayList<Usuario> usuarios = usuarioDAO.getAllUsuarios();
      if(usuarios == null || usuarios.size() == 0)
         return "nao ha usuarios";

      boolean msgsEnviadas = false;
      for(String nome : nomes) {
         for (Usuario usuario : usuarios) {
            if(usuario.getNome().equals(nome)) {
               String mensagem = criptografia.criptografar("mensagem secreta");
               boolean foiEnviado = servidorEmail.enviar("TO: " + usuario.getEmail() +
                                                                  " " + mensagem);
               if(foiEnviado) {
                  msgsEnviadas = true;
                  break;
```

```
                }
                else
                    return "servidor de email offline";
            }
        }
    }

    if(msgsEnviadas)
        return "mensagens enviadas";
    else
        return "usuarios nao encontrados";
  }
}
```

Elabore casos de teste usando JUnit e Mockito de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (*branch coverage*). Desenhe o grafo de fluxo de controle (GFC) para o método “enviarPara(..)” da classe Emissario e comente cada caso de teste com o caminho que foi executado de acordo com o GFC. O desenho do GFC pode ser feito em uma folha em branco ou encaminhada como uma imagem junto com o projeto; enumere os nós para identificar corretamente os caminhos.

(Exercício 8) Implemente casos de teste em JUnit com o intuito de alcançar 100% de cobertura do critério de teste estrutural todos-arcos (*branch coverage*). Desenhe o grafo de fluxo de controle (GFC) para seus métodos e comente cada caso de teste com o caminho que foi executado de acordo com o GFC.

```
/** extraído de G. Fraser and A. Arcuri, “Whole Test Suite Generation,” IEEE Transactions on Software Engineering, vol. 39, iss. 2, pp. 276-291, 2013. **/

public class Stack {
  int[] values = new int[3];
  int size = 0;

  void push(int x) {
    if(size >= values.length)
      resize() ;
   if(size < values.length)
     values[size++] = x;
  }

  int pop() {
    if(size > 0)
      return values[size−−];
    else
      throw new EmptyStackException();
  }

  private void resize(){
    int[] tmp = new int[values.length * 2];
    for(int i = 0; i < values.length; i++)
      tmp[i] = values[i];
    values = tmp;
  }
}
```

(Exercício 9) Considere a classe a seguir:

```
public class Calculadora {
   /**
    * @param n - inteiro
    * @param valorMaximo - valor maximo que pode ter o somatorio
    * @return - o somatorio de 0 ate |n|, caso somatorio seja <= valorMaximo
    * @throws Exception - caso o somatorio seja > valorMaximo
    */
   public int somatoriaLimitada(int n, int valorMaximo) throws Exception {
      int resultado = 0, i = 0;
      if(n < 0) {
         n = -n;
      }

      while(i<=n && resultado <= valorMaximo) {
         resultado = resultado + i;
         i++;
      }

      if(resultado > valorMaximo)
         throw new Exception("valor maximo foi ultrapassado");
      else
         return resultado;
   }
}
```

Elabore casos de teste em JUnit de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (branch coverage). Desenhe o grafo de fluxo de controle (GFC) para o método e comente cada caso de teste com o caminho que foi executado de acordo com o GFC. O desenho do GFC pode ser feito no espaço em branco no final da avaliação ou encaminhada como uma imagem junto com o projeto; enumere os nós para identificar corretamente os caminhos.

(Exercício 10) Considere as duas classes e as duas interfaces a seguir:

Usuario.java

```
public class Usuario {

        private String nome, senha,
senhaConfirmada;

        public String getNome() {
                return nome;
        }

        public void setNome(String nome) {
```

UsuarioDAO.java

```
public interface UsuarioDAO {
        public boolean
                existe(String nomeDoUsuario);
}
```

SenhaValidator.java

```
public interface SenhaValidator {
        public boolean verificar(String senha);
}
```

```
		this.nome = nome;
	}

	public String getSenha() {
		return senha;
	}

	public void setSenha(String senha) {
		this.senha = senha;
	}

	public String getSenhaConfirmada() {
		return senhaConfirmada;
	}

	public void setSenhaConfirmada(String
				senhaConfirmada)
{
		this.senhaConfirmada =
				senhaConfirmada;
	}
}
```

UsuarioValidator.java

```
public class UsuarioValidator {
	private UsuarioDAO usuarioDao;
	private SenhaValidator senhaValidator;

	public UsuarioValidator(UsuarioDAO usuarioDao) {
		this.usuarioDao = usuarioDao;
	}

	public void setSenhaValidator(SenhaValidator senhaValidator) {
		this.senhaValidator = senhaValidator;
	}

	public boolean ehUsuarioValido(Usuario u) throws Exception {
		if(u.getNome().length() <= 3)
			throw new Exception("nome do usuario precisa de pelo menos 3 caracteres");

		if(! u.getSenha().equals(u.getSenhaConfirmada()))
			throw new Exception("senhas diferentes");

		if(usuarioDao.existe( u.getNome() ))
			throw new Exception("usuario ja existe");

		if(senhaValidator.verificar( u.getSenha() ))
			throw new Exception("senha invalida");

		String nome = u.getNome();
		boolean isValid = true;
		for(int i = 0; i < nome.length(); i++) {
			char atual = nome.charAt(i);
			if(!Character.isAlphabetic(atual) && !Character.isDigit(atual))
				isValid = false;
		}
```

```
                return isValid;
        }

}
```

(a) Desenhe o grafo de fluxo de controle (GFC) para o método "ehUsuarioValido(..)" da classe UsuarioValidator. O desenho do GFC pode ser feito em uma folha em branco ou encaminhada como uma imagem junto com o projeto; enumere os nós para identificar corretamente os caminhos.

(b) Elabore casos de teste usando JUnit e Mockito de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (*branch coverage*).

(c) Comente cada caso de teste com o caminho que foi executado de acordo com o GFC.

(Exercício 11) Considere a interface e a classe a seguir:

RedeMovel.java

```
public interface RedeMovel {
    public boolean estaConectado();
    public boolean enviarSMS(String telefone, String mensagem);
}
```

ContadorNaRede.java

```
public class ContadorNaRede {
    RedeMovel rede;

    public ContadorNaRede(RedeMovel rede) {
        this.rede = rede;
    }

    public String enviarNumeros(String telefone, int v[ ]) {
        int npar = 0, nNegativo = 0;
        for (int i = 0; i < v.length; i++) {
            if(v[i] % 2 == 0)
                npar++;

            if(v[i] < 0)
                nNegativo++;
        }
        String msg = "Enviado. pares: " + npar + "; negativos: " + nNegativo;

        if(rede == null || ! rede.estaConectado())
            throw new RuntimeException("Sem internet");

        if(rede.enviarSMS(telefone, msg))
            return msg;
        else
            return "Erro no envio. Verifique.";
    }
}
```

Elabore casos de teste em JUnit de forma a alcançar 100% de cobertura do critério de teste caixa-branca todos-arcos (*branch coverage*). Desenhe o grafo de fluxo de controle (GFC) para o método “enviarNumeros(..)” e comente cada caso de teste com o caminho que foi executado de acordo com o GFC. Enumere os nós para identificar corretamente os caminhos.

(Exercício 12) Considere a interface e a classe a seguir:

CartorioEleitoral.java

```
public interface CartorioEleitoral {

    /** Esse método pode retornar:
     * - "nao existe": se o cpf não possui
          titulo associado
     * - "pendencia": o titulo possui
          alguma pendencia
     * - "OK": situacao regularizada
          para o título  */

    public String verificar(String cpf);

}
```

VerificadorEleitoral.java

```
public class VerificadorEleitoral {
    private CartorioEleitoral cartorioEleitoral;

    public VerificadorEleitoral(CartorioEleitoral cartorioEleitoral) {
        this.cartorioEleitoral = cartorioEleitoral;
    }

    public String consultarSituacao(int idade, String cpf) throws Exception {
        if(idade < 0 || idade > 200) throw new Exception("idade invalida");

        if(cpf == null || cpf.length() != 11) throw new Exception("cpf invalido");

        if(idade < 16) return "nao pode votar";

        String status = cartorioEleitoral.verificar(cpf);
        if(status.equals("nao existe"))
            return "faca um titulo";
        else if(status.equals("pendencia"))
            return "regularize seu titulo";
        else if(status.equals("OK")) {
            String ret = "voto obrigatorio";
            if(idade <= 17 || idade > 70)
                ret = "voto facultativo";

            return ret;
        }
        return "erro desconhecido";
    }
}
```

Elabore casos de teste em JUnit e Mockito de forma a alcançar 100% de cobertura do critério de teste caixa-branca todos-arcos (*branch coverage*). Desenhe o grafo de fluxo de controle (GFC) para o método “consultarSituacao(..)” e comente cada caso de teste com o caminho que foi executado de acordo com o GFC. O desenho do GFC pode ser feito em uma folha em branco ou como uma imagem junto com o projeto; enumere os nós no código para identificar corretamente os caminhos.

(Exercício 13) Considere a classe a seguir:

Lambda.java

```
public class Lambda {
	/**
	 * @param strings
	 * @return um array com 2 inteiros contando quantas palavras
	 * comecam com 'a' e 'z', respectivamente
	 */
	public int[] countWordsStartingWithAandZ(List<String> strings) {
		int[] ret = {0, 0};

		strings.stream().forEach(s -> {
			if(s.startsWith("a"))
				ret[0]++;

			if(s.startsWith("z"))
				ret[1]++;
		});

		return ret;
	}
}
```

(a) Desenhe o grafo de fluxo de controle (GFC) para o método "countWordsStartingWithAandZ(..)" da classe UsuarioValidator.
Obs. *Na construção do GFC, considere o forEach como um loop while.*

(b) Elabore casos de teste usando JUnit de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (*branch coverage*).

(c) Comente cada caso de teste com o caminho que foi executado de acordo com o GFC.

(Exercício 14) Considere a interface e as 2 classes a seguir:

UsuarioDAO.java
```
public interface UsuarioDAO {
    public List<Usuario> getUsuarios();
}
```

Usuario.java
```
public class Usuario {
    private String nome, senha;

    public Usuario(String pNome, String pSenha) {
        nome = pNome;
        senha = pSenha;
    }

    public String getNome() {
        return nome;
    }

    public String getSenha() {
        return senha;
    }
}
```

LoginController.java
```
public class LoginController {
    private UsuarioDAO usuarioDAO;

    public void setUsuarioDAO(UsuarioDAO usuarioDAO) {
        this.usuarioDAO = usuarioDAO;
    }

    public boolean efetuarLogin(String nome, String senha)
                                                    throws Exception {
        if(nome.equals("") || senha.equals(""))
            throw new Exception("preencher nome e senha");

        List<Usuario> usuarios = usuarioDAO.getUsuarios();
        Usuario u = null;
        for (int i = 0; i < usuarios.size(); i++) {
            if(usuarios.get(i).getNome().equals(nome)) {
                u = usuarios.get(i);
                break;
            }
        }
        if(u == null)
            throw new Exception("usuario nao existe");
        else if(u.getSenha().equals(senha))
            return true;
        else
            return false;
    }
}
```

Elabore casos de teste em JUnit e Mockito de forma a alcançar 100% de cobertura do critério de teste caixa-branca todos-arcos (*branch coverage*). Desenhe o grafo de fluxo de controle (GFC) para o método “efetuarLogin(..)” e comente cada caso de teste com o caminho que foi executado de acordo com o GFC. O desenho do GFC pode ser feito em uma folha em branco com seu nome ou encaminhada como uma imagem; enumere os nós no código para identificar corretamente os caminhos.

(Exercício 15) Considere as duas classes e as duas interfaces a seguir:

Empresa.java
```
public class Empresa {
    private int id;
    private String nome, cidade;

    public Empresa(int id, String nome, String cidade) {
        this.id = id;
        this.nome = nome;
        this.cidade = cidade;
    }

    public int getId() {
        return id;
    }

    public String getNome() {
        return nome;
    }

    public String getCidade() {
        return cidade;
    }

}
```

EmpresaDAO.java
```
public interface EmpresaDAO {

    public List<Empresa> getAll();

}
```

Log.java
```
public interface Log {

    public void realizarBusca(String string);

}
```

Buscador.java

```
public class Buscador {

   private EmpresaDAO empresaDao;
   private Log log;

   public Buscador(EmpresaDAO empresaDao, Log log) {
      this.empresaDao = empresaDao;
      this.log = log;
   }

   public Empresa achar(String campo, String valor, boolean exato)  throws Exception {
      log.realizarBusca(campo + "," + valor + "," + exato);

      if(!campo.equals("nome") && !campo.equals("cidade"))
         throw new Exception("campo invalido");

      List<Empresa> empresas = empresaDao.getAll();
      for (int i = 0; i < empresas.size(); i++) {
         Empresa e = empresas.get(i);
         String valorEmp;
         if(campo.equals("nome"))
            valorEmp = e.getNome();
         else
            valorEmp = e.getCidade();

         if(exato && valorEmp.equals(valor))
               return e;
         else if(valorEmp.contains(valor))
               return e;
      }

      return null;
   }

}
```

(a) Desenhe o grafo de fluxo de controle (GFC) para o método “achar(..)” da classe Buscador. O desenho do GFC pode ser feito em uma folha em branco ou encaminhada como uma imagem junto com o projeto; enumere os nós no código-fonte para identificar corretamente os caminhos.

(b) Elabore casos de teste usando JUnit e Mockito de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (*branch coverage*).

(c) Comente cada caso de teste com o caminho que foi executado de acordo com o GFC.

(Exercício 16) Considere as duas classes e a interface a seguir:

Cliente.java

```
public class Cliente {
            private int codigo;
            private String nome;
            private int desconto = 0;

            public Cliente(int codigo, String nome) {
                        this.codigo = codigo;
                        this.nome = nome;
            }

            public int getCodigo() {  return codigo;  }

            public String getNome() {  return nome;  }
```

ClienteRepo.java

```
public interface ClienteRepo {
            public Cliente getCliente(int codigo);
}
```

```
	public int getDesconto() {  return desconto;  }

	public void setDesconto(int desconto) {
		this.desconto = desconto;
	}
}
```

Resolvedor.java

```
public class Resolvedor {
	private ClienteRepo repo;

	public Resolvedor(ClienteRepo repo) {  this.repo = repo;  }

	public ArrayList<Cliente> definirPromocao(int[ ] cods) throws Exception {
		if (cods == null || cods.length == 0)
			throw new IllegalArgumentException("sem codigo algum");

		var clientes = new ArrayList<Cliente>();
		for (int cod : cods) {
			Cliente c = repo.getCliente(cod);
			if (c == null)
				throw new Exception("Codigo do cliente nao valido");

			clientes.add(c);
		}

		var resposta = new ArrayList<Cliente>();
		if (clientes.size() >= 3) {
			// todos ganham 25% de desconto
			for (Cliente c : clientes) {
				c.setDesconto(25);
				resposta.add(c);
			}
		}
		else {
			// o 1º cliente ganha 15%
			clientes.get(0).setDesconto(15);
			resposta.add(clientes.get(0));

			//e, se existir, o 2º ganha 10%
			if (clientes.size() > 1) {
				clientes.get(1).setDesconto(10);
				resposta.add(clientes.get(1));
			}
		}

		return resposta;
	}
}
```

(a) Desenhe o grafo de fluxo de controle (GFC) para o método “definirPromocao(..)” da classe Resolvedor. O desenho do GFC pode ser feito em como uma imagem encaminhada junto com o projeto; enumere os nós no código-fonte para identificar corretamente os caminhos.

(b) Elabore casos de teste usando JUnit e Mockito de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (*branch coverage*).

(c) Comente cada caso de teste com o caminho que foi executado de acordo com o GFC.

(Exercício 17) Considere as três interfaces e a classe a seguir:

Coder1.java

```
public interface Coder1 {
	public boolean m1(ArrayList<String> a);
}
```

Coder2.java

```
public interface Coder2 {
	public boolean m2(ArrayList<String> a);
}
```

Coder3.java

```
public interface Coder3 {
	public int m3();
}
```

AClass.java

```
public class AClass {
	private Coder1 coder1;
	private Coder2 coder2;
	private Coder3 coder3;

	public AClass(Coder1 coder1, Coder2 coder2, Coder3 coder3) {
		this.coder1 = coder1;
		this.coder2 = coder2;
		this.coder3 = coder3;
	}

	public String cifrar(ArrayList<String> msg) {
		if (msg == null || msg.size() == 0 || msg.contains("STOP"))
			return "INVALID";

		if (coder1.m1(msg) && coder2.m2(msg))
			return "C1-C2";

		var count = 0;
		for (String s : msg) {
			if (s.equals("HI"))
				count++;
		}
		if (count >= 2)
			return "2 OR MORE HIs";

		int num = coder3.m3();
		if (num > 1) {
			String ret = "";
			for (int i = 1; i <= num; i++)
				ret += "-" + msg.get(0);

			return ret;
		}

		return msg.get(0);
	}
}
```

(a) Desenhe o grafo de fluxo de controle (GFC) para o método “cifrar(..)” da classe AClass. O desenho do GFC pode ser feito em como uma imagem encaminhada junto com o projeto; enumere os nós no código-fonte para identificar corretamente os caminhos.

(b) Elabore casos de teste usando JUnit e Mockito de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (*branch coverage*).

(c) Comente cada caso de teste com o caminho que foi executado de acordo com o GFC.

(Exercício 18) Considere as duas classes e as 4 interfaces a seguir:

Order.java

```
public class Order {
        private int code;
        private String email, desc, address;
        List<Integer> prodIds;

        public Order(int code, String email, String desc,
                String address) {
                this.code = code;
                this.email = email;
                this.desc = desc;
                this.address = address;
                prodIds = new ArrayList<Integer>();
        }

        public int getCode() { return code; }
        public String getEmail() { return email; }
        public String getDesc() { return desc; }
        public String getAddress() { return address; }
        public List<Integer> getProdIds() { return prodIds; }
}
```

EmailSender.java

```
public interface EmailSender {
        public boolean isOffline();

        public int sendEmail(String email,
                        String title, String body);
}
```

Repository.java

```
public interface Repository {
        boolean orderProduct(int prodId);
}
```

TransportService.java

```
public interface TransportService {
        public boolean isDown();

        public int
            makeTag(int code, String address);
}
```

Validator.java

```
public interface Validator {
        public List<String>
            validateBasicData(Order customer);
}
```

ProcessOrderUC.java

```
public class ProcessOrderUC {

        private Validator validator;
        private TransportService service;
        private EmailSender emailSender;
        private Repository repo;

        public ProcessOrderUC(Validator validator, Repository repo) {
                this.validator = validator;
                this.repo = repo;
        }

        public void setService(TransportService service) {
                this.service = service;
        }

        public void setEmailSender(EmailSender emailSender) {
                this.emailSender = emailSender;
        }

        public int[] process(Order order) {
                var errors = validator.validateBasicData(order);
                if (!errors.isEmpty()) {
                        var errorMsg = String.join(",", errors);
                        throw new IllegalArgumentException(errorMsg);
                }

                if (service.isDown() || emailSender.isOffline()) {
                        throw new RuntimeException("Services offline. Try again later.");
                }

                int orderedProds = 0, unorderedProds = 0;
                for (int prodId : order.getProdIds()) {
                        var success = repo.orderProduct(prodId);
                        if (success) {
                                orderedProds++;
                        } else {
                                unorderedProds++;
                        }
                }

                var transportId = service.makeTag(order.getCode(), order.getAddress());
                var emailId = emailSender.sendEmail(order.getEmail(), "Your order", order.getDesc());
                int[] ret = { transportId, emailId, orderedProds, unorderedProds };
```

```
                return ret;
        }

}
```

(a) Desenhe o grafo de fluxo de controle (GFC) para o método “process(..)” da classe ProcessOrderUC. O desenho do GFC pode ser feito em como uma imagem encaminhada junto com o projeto; enumere os nós no código-fonte para identificar corretamente os caminhos.

(b) Elabore casos de teste usando JUnit e Mockito de forma a alcançar 100% de cobertura do critério de teste estrutural todos-arcos (*branch coverage*).

(c) Comente cada caso de teste com o caminho que foi executado de acordo com o GFC.